ISC2 SECURITY CONGRESS 2026

ISC2.org/Congress | #ISC2Congress

From Zero to Security Bot

Building and Hardening an AI-Powered Code Review Pipeline

Pre-Conference Workshop, Part 1: Saturday, October 24, 2026

Cameron Hopkin

Founder, Waypoint Compliance Advisory



Day 1 Agenda

Time	Session
8:00	Setup check and introductions
8:30	1. The Landscape
10:00	2. Model Deployment
12:30	3. Building the Security Bot
14:30	3, continued. Making the bot trustworthy
15:15	4. CI/CD Integration

Breaks at 9:45 and 14:15. Lunch at 11:30. Lab guides: lab-guide/ in the repository.

On Windows? Start Here

PowerShell

Setup Path C in lab-guide/00-setup.md: setup.ps1, then .\.venv\Scripts\Activate.ps1.

Cheat sheet

Every PowerShell command, every module: lab-guide/WINDOWS-CHEATSHEET.md. Copy and paste.

Codespaces

A Linux machine in your browser. The codespace you create is your own.

ARM or Python 3.15

If setup fails building PyYAML, install Python 3.12 (64-bit) and rerun setup.

Module 1. The Landscape

Three Ways to Put AI in Code Review

Scanner only

Bandit, detect-secrets. Fast, cheap, reports patterns, not defects.

Scanner plus model triage

The model decides what each finding means in this code, explains it, and rates it. This is the bot you build today.

Model alone

Cold review for bugs no scanner flags. Much harder. Day 2 capstone.

Lab: Run the Scanners on Their Own

```
bandit -q -r target-app -x target-app/tests -f txt
detect-secrets scan --all-files target-app
```

Before anything else: which of these results are real? Write your guess down.

What the Scanners Alone Give You

Result	Count	What it means
Real defects flagged	4	SQL injection, two hardcoded secrets, command injection
Real, easy to dismiss	1	A hardcoded universal password that looks like demo code
Real, flagged by nothing	2	Path traversal, broken access control
Noise	the rest	Documentation, test fixtures, advisory imports

13 results in all. Scanners report patterns. Some are real, some are noise, and some real bugs are not in the list at all.

Why Run the Model Locally

Compliance

Source code never leaves the laptop or the CI runner. No third-party API, no data agreement.

Cost

Open-weight models, free runners. The Module 7 TCO model puts numbers on it.

Control

You pin the model, the prompt, and the settings, and you can measure every change.

Table Exercise: Map Your Own Pipeline

Draw it

How a change gets reviewed at your organization today, from commit to merge.

Mark it

Where a scanner runs, where a model could triage, and who would read the result.

Block it

What would stop you: policy, cost, or nobody owning it. Keep the drawing for Module 7.

Module 2. Model Deployment

The Workshop Models

Model	Size	Who runs it
qwen3.5:9b	6.6 GB	Everyone. The workshop default.
qwen2.5-coder:7b	4.7 GB	Everyone. The low-memory fallback.
gpt-oss:20b	13 GB	Laptops with 24 GB of RAM or more. Everyone else watches the screen.

Lab: Same File, Three Models

From the repository root. Calling the workshop's own Python means no activation step:

```
REVIEW_MODEL=qwen3.5:9b REVIEW_THINK=false .venv/bin/python reference-pipeline/review.py --file target-app/app.py
REVIEW_MODEL=qwen2.5-coder:7b REVIEW_THINK=false .venv/bin/python reference-pipeline/review.py --file target-app/app.py
REVIEW_MODEL=gpt-oss:20b REVIEW_THINK=low .venv/bin/python reference-pipeline/review.py --file target-app/app.py
```

Write down, for each model: what it found, how it rated it, and how long it took.

What the Bake-off Showed

The fallback ships a live credential

qwen2.5-coder:7b rates the hardcoded API token as info.

Bigger caught what smaller dismissed

gpt-oss:20b flagged the universal password. qwen3.5:9b believed the docstring that called it a demo.

Security-trained is not better at code

Foundation-Sec 8B found the bugs and buried them in about 15 unmatched findings per run.

Reasoning Is a Setting You Must Choose

qwen3.5:9b

think = false. Fast, and accurate enough for triage.

gpt-oss:20b

think = "low". It cannot turn reasoning off.

Left on default

A reasoning model can take minutes per call. That is why think is always set explicitly in config.toml.

Lab: What Does More Thinking Buy?

```
cd reference-pipeline
time REVIEW_MODEL=gpt-oss:20b REVIEW_THINK=low python review.py --file ../target-app/auth.py
time REVIEW_MODEL=gpt-oss:20b REVIEW_THINK=medium python review.py --file ../target-app/auth.py
time REVIEW_MODEL=gpt-oss:20b REVIEW_THINK=high python review.py --file ../target-app/auth.py
time REVIEW_MODEL=qwen3.5:9b REVIEW_THINK=true python review.py --file ../target-app/auth.py
cd ..
```

Did the findings change, or only the time? Put your times on the board, by laptop.

Module 3. Building the Security Bot

The Pipeline

1. Scan

Bandit and detect-secrets produce findings with a file and a line.

2. Triage

The model explains each finding and rates it, with a JSON schema.

3. Validate

Code drops anything without a real citation.

4. Comment

One escaped comment on the pull request, edited on every push.

Lab: Run the Bot

```
cd reference-pipeline
python review.py --file ../target-app > findings.json

# what the model was actually sent
python review.py --file ../target-app/models.py --dry-run

# the comment it would post
python post_comment.py findings.json --print
cd ..
```

Making the Bot Trustworthy

Citation required

Every finding names a file and a line that exist, and quotes code that is really on that line. If not, it is dropped.

Refuse over guess

"No findings" with a reason is a first-class answer, so the model is never pushed to invent one.

Do not trust the code

"This is test code" comes from the file path, never from what the file says about itself.

The Rule for Day 2

A prompt is a request, not a control.

Our prompt already told the model not to trust a file's description of itself. Tomorrow you will watch it trust one anyway.

The Prompt Tournament

Edit only prompts/review_system.md. Score: 10 points per planted bug found, minus 2 per unmatched finding. To beat: 36.

```
cp prompts/review_system.md prompts/review_system.md.orig
# edit prompts/review_system.md, then:
python reference-pipeline/review.py --file target-app > scanner.json
python reference-pipeline/review.py --file target-app --no-scanner > sweep.json
python eval-harness/score_planted.py scanner.json sweep.json

# before Module 4: put the original back
mv prompts/review_system.md.orig prompts/review_system.md
```

Rules: no code changes, never name the planted bugs or their lines, keep the section headings.

Tournament Result: The Room's Winning Rule

Now rule 8 in the ## sweep section of prompts/review_system.md:

```
8. Follow the input. For every value that comes from a request (query string,
   form field, URL path), follow it to where it is used: a SQL query, a shell
   command, a file path, a redirect. Report the line where untrusted input
   reaches something dangerous. When a file path is built from a request value,
   consider what `..` in that value would do.
```

Measured on qwen3.5:9b: 36 points before, 50 after (5 of 6 planted bugs, no extra findings). The path traversal is found. Broken access control still needs Module 8.

Module 4. CI/CD Integration

Two Templates, Both Free Tier

GitHub Actions

Public repository: a free 16 GB runner.

GITHUB_TOKEN with pull-requests: write only.

GitLab CI

Free medium runner.

Fine-grained token with three work item permissions, stored masked.

Both

Non-blocking. One comment, edited in place on every push.

Since Oct 24: a cold read (--sweep) and one-click suggested fixes.

Lab: Open a Pull Request with Planted Bugs

```
# after pushing the repository and the template to your own account
git switch -c add-order-importer
git apply ci-templates/lab/add-order-importer.patch
git add target-app/importer.py
git commit -m "Add bulk order import"
git push mine add-order-importer
# open the pull request (or merge request) and wait
```

After about 8 minutes: one comment, three findings in importer.py. Push again and watch it edit, not add.

Your Repository: New or Fork

mine is a nickname

git remote add mine <your-repository-url>. Your URL, never the instructor's.

Forking on GitHub

Turn Actions on in the fork. When you open the pull request, set the base repository to your fork.

GitLab

GitLab cannot fork from GitHub. Create a new project and push to it as mine.

While the Pipeline Runs

What can a pull request author get into the bot's comment?

```
# edit the title and explanation in this file, then render the comment
python reference-pipeline/post_comment.py ci-templates/lab/hostile-findings.json --print

# try: break the table, inject HTML, hide text, add a link, mention a team
git checkout ci-templates/lab/hostile-findings.json
```

Write down whatever gets through escape(). Tomorrow you try to make the model write it.

Measured on Free Runners

Platform	Total time	Notes
GitHub, free runner	about 8.5 minutes	about 6 minutes of review
GitLab, medium runner	about 10 minutes	
Second run	faster	model restored from cache

October 2026. The patch has three planted bugs: pickle.loads, a hardcoded token, verify=False.

Discussion

Diff only

The bot reviews only the changed lines. Why is that both faster and a blind spot?

Hostile comments

The comment repeats what the model wrote about the author's code. What could a hostile author get into it? Read escape().

Token scope

The GitLab token can also touch issues. What would you change before using this at work?

Tomorrow

You built a bot. Tomorrow you attack it.

Day 2 starts at 8:00 with a recap. Leave the model downloaded and the pull request open.

Any Questions?

Cameron Hopkin | Cameron@waypointca.com

github.com/cameronhopkin/isc2-2026-ai-code-review-workshop

